

第11章 蓄势待发

前缀和、差分与双指针

JD121：蓄势之术

AcWing 795 | JD121

蓄势阁中，赵晴儿盘膝而坐，面前摆着一排石块，每块上刻着一个数字。

"如果我问你第3块到第7块的总和，你怎么算？"赵晴儿问。

李少白："从第3块加到第7块，五次加法。"

"如果问一千次呢？"赵晴儿摇头，"每次从头算太浪费了。提前算好从第1块到每一块的累积和——第1块到第7块的和减去第1块到第2块的和，就是第3块到第7块的和。一次减法就够了。"

"这就是前缀和——蓄势于前，取用于后。"

给定一个长度为 n 的整数序列和 m 个查询，每个查询求 $[l, r]$ 的区间和。

输入格式

第一行两个整数 n 和 m 。第二行 n 个整数。接下来 m 行，每行两个整数 l 和 r （从1开始计数）。

输出格式

m 行，每行一个整数。

输入样例

```
5 3
2 1 3 6 4
1 3
2 4
1 5
```

输出样例

```
6
10
16
```

解题思路：

前缀和： $s[i] = a[1] + a[2] + \dots + a[i]$ 。区间和 = $s[r] - s[l-1]$ 。

► 参考代码

JD122：方阵蓄势

AcWing 796 | JD122

赵晴儿在沙盘上画了一个 n 行 m 列的数字矩阵。"如果我问你左上角 $(x1,y1)$ 到右下角 $(x2,y2)$ 这个子矩阵里所有数的和，你怎么算？"

李少白想逐行累加。赵晴儿摇头："太慢了。和一维前缀和一样的思路——提前算好从 $(1,1)$ 到每个位置的累积和。查询时用容斥原理：大矩形减去左边和上边的多余部分，加上左上角多减的部分。"

"蓄势蓄到二维，一击覆盖任意子矩阵。"

给定一个 $n \times m$ 的整数矩阵和 q 个查询，每个查询求子矩阵的和。

输入格式

第一行三个整数 n 、 m 和 q 。接下来 n 行，每行 m 个整数。接下来 q 行，每行四个整数 $x1$ 、 $y1$ 、 $x2$ 、 $y2$ 。

输出格式

q 行，每行一个整数。

输入样例

```
3 4 2
1 3 4 5
2 6 9 4
1 4 7 5
1 1 2 2
2 1 3 3
```

输出样例

```
18
43
```

解题思路：

二维前缀和： $s[i][j]$ = 左上角到 (i,j) 的累加。子矩阵和 = $s[x2][y2] - s[x1-1][y2] - s[x2][y1-1] + s[x1-1][y1-1]$ 。

► 参考代码

JD123：差分之术

AcWing 797 | JD123

赵晴儿指着一排石块："我要把第3块到第7块每块都加上5。如果一块块改，太慢了。"

梁嘉峰说："差分——前缀和的逆运算。你在第3块的位置加5，在第8块的位置减5。最后对整排求一次前缀和，第3到第7块就都加了5，其余不变。"

"就像往水面扔两块石头，"李少白恍然大悟，"波纹叠加后就是最终结果。"

给定一个长度为 n 的整数序列和 m 个操作，每个操作将 $[l, r]$ 的每个数加 c 。输出所有操作后的序列。

输入格式

第一行两个整数 n 和 m 。第二行 n 个整数。接下来 m 行，每行三个整数 l 、 r 和 c 。

输出格式

一行，操作后的序列，空格隔开。

输入样例

```
1 2 3 4 5
1 3 2
2 4 1
3 5 3
```

输出样例

```
3 5 8 8 8
```

解题思路：

差分： $b[l] += c$, $b[r+1] -= c$ 。所有操作完成后，对 b 求前缀和得到最终序列。

► 参考代码

JD124：方阵差分

AcWing 798 | JD124

赵晴儿在沙盘上画了一个 n 行 m 列的矩阵："我要把左上角 $(x1,y1)$ 到右下角 $(x2,y2)$ 这个子矩阵里每个数都加上 c 。"

梁嘉峰说："和一维差分一样的思路——在差分矩阵的四个角加减，最后对整个矩阵求一次二维前缀和，就还原出最终结果了。"

"蓄势蓄到二维，差分也能覆盖任意子矩阵。"

给定一个 $n \times m$ 的整数矩阵和 q 个操作，每个操作将子矩阵 $(x1,y1)-(x2,y2)$ 的每个元素加 c 。输出所有操作后的矩阵。

输入格式

第一行三个整数 n 、 m 和 q 。接下来 n 行，每行 m 个整数。接下来 q 行，每行五个整数 $x1$ 、 $y1$ 、 $x2$ 、 $y2$ 和 c 。

输出格式

n 行，每行 m 个整数，空格隔开。

输入样例

```
3 4 2
1 2 3 4
5 6 7 8
9 10 11 12
1 1 2 2 1
2 2 3 3 2
```

输出样例

```
2 3 3 4
6 10 10 8
9 12 14 12
```

解题思路：

二维差分：在 $(x1,y1)+c$, $(x2+1,y1)-c$, $(x1,y2+1)-c$, $(x2+1,y2+1)+c$ 。操作完后求二维前缀和。

► 参考代码

JD125：离散聚力

AcWing 802 | JD125

赵晴儿指着一条无限长的数轴："坐标从负无穷到正无穷，但真正有值的位置只有几个。如果开一个那么大的数组，太浪费了。"

"所以要把这些稀疏的大坐标映射到连续的小下标上——这就是离散化。"梁嘉峰接话，"映射之后再使用前缀和，就能 $O(1)$ 回答区间求和的查询。"

"先聚力于有用之处，再一击命中。"

假定有一个无限长的数轴，初始每个坐标上都是0。先进行 n 次操作，每次将位置 x 上的数加 c 。然后进行 m 次查询，每次求区间 $[l, r]$ 的和。

输入格式

第一行两个整数 n 和 m 。

接下来 n 行，每行两个整数 x 和 c 。

接下来 m 行，每行两个整数 l 和 r 。

输出格式

m 行，每行一个整数。

输入样例

3 3

1 2

3 6

7 5

1 3

4 6

7 8

输出样例

8

0

5

解题思路：

离散化：排序去重后映射到连续下标。用前缀和回答区间查询。

► 参考代码

JD126：合围归阵

AcWing 803 | JD126

梁嘉峰在地上画了一堆区间，有些重叠，有些相邻。"把这些区间合并——有交集的归为一组，最后数一数有几组。"

赵晴儿蹲下来看了一会儿："先按左端点从小到大排。然后从左往右扫——如果当前区间的左端点在前一个区间内，说明它们重叠，合并；否则新开一组。"

"合围之势，聚散有序。"

给定 n 个区间 $[l_i, r_i]$ ，合并所有有交集的区间（端点相交也算），输出合并后的区间个数。

输入格式

第一行一个整数 n ($1 \leq n \leq 100000$) 。

接下来 n 行，每行两个整数 l_i 和 r_i 。

输出格式

一行，合并后的区间个数。

输入样例

```
5
1 2
2 4
5 6
7 8
7 10
```

输出样例

```
3
```

解题思路：

按左端点排序，遍历合并：当前区间左端点 $\leq$ 前一个右端点则合并，否则新开。

► 参考代码

JD127：一箭穿心

AcWing 905 | JD127

赵晴儿在数轴上标了 N 个区间，每个区间表示一个敌人的巡逻范围。"用最少的暗器，让每个敌人的巡逻范围里至少中一枚。"

李少白想每个区间放一枚。赵晴儿摇头："贪心——按右端点从小到大排。第一枚暗器射在第一个区间的右端点，这样所有包含这个点的区间都中了。然后跳过这些区间，再射下一批。"

"选右端点是因为它最'靠左'——能覆盖尽可能多的后续区间。"

给定 N 个闭区间 $[a_i, b_i]$ ，选择尽量少的点，使得每个区间内至少有一个点。

输入格式

第一行一个整数 N ($1 \leq N \leq 100000$)。接下来 N 行，每行两个整数 a_i 和 b_i 。

输出格式

一行，最少需要的点数。

输入样例

```
3
1 3
2 4
3 5
```

输出样例

```
1
```

解题思路：

按右端点排序，每次选当前区间的右端点，跳过所有包含该点的区间。

► 参考代码

JD128：合果成堆

AcWing 148 | JD128

果园里有 N 堆果子，每堆重量不同。赵晴儿要把它们合并成一堆，每次只能合并两堆，消耗的能量等于两堆重量之和。

"如果先合并重的，后面每次合并都要带着这堆重的走，代价越来越大。"赵晴儿说，"所以反过来——每次选最轻的两堆合并，这样每一步的代价都最小。"

李少白想到用小根堆："每次取出堆顶（最轻的两堆），合并后放回去。总代价最小。"

这就是哈夫曼树的思想。

有 N 堆果子，每次合并两堆（代价为两堆之和），求合并为一堆的最小总代价。

输入格式

第一行一个整数 N ($1 \leq N \leq 10000$)。第二行 N 个整数，表示每堆的重量。

输出格式

一行，最小总代价。

输入样例

```
3
1 2 9
```

输出样例

```
15
```

解题思路：

小根堆：每次取最小的两个合并，结果放回堆中。 $O(n\log n)$ 。

► 参考代码

JD129：中位之选

AcWing 104 | JD129

数轴上有 N 个商铺，赵晴儿要建一个货仓，让所有商铺到货仓的货物运输距离之和最小。

"选在哪里？"李少白问。

"排序后取中位数。"赵晴儿说，"数学可以证明：中位数使绝对偏差之和最小。如果选在中位数左边，右边的商铺距离全部变远；选在中位数右边，左边的商铺距离全部变远。只有中位数是平衡点。"

给定 N 个点的坐标，求一个位置使所有点到该位置的绝对距离之和最小，输出最小距离之和。

输入格式

第一行一个整数 N ($1 \leq N \leq 100000$)。第二行 N 个整数，表示各点坐标。

输出格式

一行，最小距离之和。

输入样例

```
5
1 2 3 4 5
```

输出样例

解题思路：

排序后取中位数（下标 $n/2$ ），计算所有点到中位数的距离之和。

► 参考代码